

II. Basic information

Please provide your preferred information for use in announcing the winners of the competition.

- Name (first and last name or first name and last initial):
 - Tuan Dung Le
- Hometown: Vietnam
- Social handle or URL (optional):
 - LinkedIn: [in/tuandungle/](https://www.linkedin.com/in/tuandungle/)

III. Model documentation and write-up

Information included in this section may be shared publicly with challenge results. You can respond to these questions in an e-mail or as an attached file. Please number your responses.

1. Who are you (mini-bio) and what do you do professionally? If you are on a team, please complete this block for each member of the team.
I'm currently PhD student in Computer Science at University of South Florida. My research focuses on applying natural language processing techniques to extract information from unstructured clinical and medical texts, especially in low-resource settings.
I also enjoy participating in AI competitions, where I can apply my skills to solve real-world problems.
2. What motivated you to compete in this challenge?
I have experience with image and text data, but I have not worked with audio data before. I saw this challenge as a valuable opportunity to expand my skill set and learn how to train and evaluate ASR models.
3. High level summary of your approach: what did you do and why?
TLDR: I train a multi-task, hybrid CTC-based ASR model that jointly optimizes standard CTC loss with a 4-way consistency objective.

I find that combining W2v-BERT 2.0 and WavLM-Large as a hybrid backbone outperforms using a single W2v-BERT model. However, making the model converge is challenging; my solution is to apply stochastic backbone drop during training.

Inspired by the CR-CTC paper, I introduce a 4-way consistency loss across different types of augmentations, internal model dropout, and noise levels, which further improves robustness.

Next, I implement multi-task learning with two CTC heads using additional labels from the Word track, with the intuition that the model can jointly learn word-level and IPA representations. However, this approach only provides modest gains.

Additionally, I leverage the large amount of unlabeled data in the Word track by generating pseudo-labels, which further improves performance when retraining on the full dataset with high-confidence pseudo-labels.

For the final submission, I ensemble predictions from four models, which provides an additional boost in overall accuracy.

4. Do you have any useful charts, graphs, or visualizations from the process?

Impact of each strategy on Public Leaderboard (from 0.2846 to 0.2618)

- Add 4-way consistency loss: ~0.015 CER reduction
- From W2v-BERT to Hybrid W2v-BERT+WavLM: ~0.002 CER reduction
- Add Multi-task training: ~0.0002 CER reduction
- Add Pseudo-labeling: ~0.002 CER reduction
- Final ensemble (3–4 models, best-score selection): ~0.003–0.004 CER reduction

5. Copy and paste the 3 most impactful parts of your code and explain what each does and how it helped your model.

First, we find that using CTC loss with “sum” reduction performs better than “mean” reduction for this task.

```
self.loss_fct = nn.CTCLoss(blank=config.pad_token_id, zero_infinity=True,  
reduction="sum")
```

Second, backbone drop rate helps stabilize training and improves convergence of my hybrid model.

```
bert_out = self.bert(input_features).last_hidden_state  
wavlm_out = self.wavlm(input_values).last_hidden_state
```

```
# Stochastic Backbone Dropping
```

```
if self.training:  
    prob = torch.rand(1, device=bert_out.device)  
    if prob <= BB_DROP:  
        bert_out = torch.zeros_like(bert_out)  
    elif prob >= (1 - BB_DROP):  
        wavlm_out = torch.zeros_like(wavlm_out)
```

```
combined = torch.cat([bert_out, wavlm_out], dim=-1)  
features = self.bridge_layer(combined)
```

Third, I implement a custom compute_loss function to incorporate the 4-way consistency loss, with separate weighting for the phonetic and word prediction heads.

```
def compute_loss(self, model, inputs, return_outputs=False, **kwargs):
    if not model.training:
        outputs = model(**inputs)
        return (outputs["loss"], outputs) if return_outputs else outputs["loss"]

    inputs_4n = {k: torch.cat([v, v], dim=0) for k, v in inputs.items()}
    outputs = model(**inputs_4n)

    p_logits, w_logits = outputs["phoneme_logits"], outputs["word_logits"]
    n = p_logits.shape[0] // 4
    seq_len = p_logits.shape[1]

    att_mask = inputs["attention_mask"][:n]
    new_lengths = (att_mask.sum(dim=-1).float() / att_mask.shape[1] *
seq_len).round().long()
    pad_mask = make_pad_mask(new_lengths, seq_len).unsqueeze(-1)

    def get_consistency(logits):
        a1, a2 = logits[0:n], logits[n:2*n]
        b1, b2 = logits[2*n:3*n], logits[3*n:4*n]
        l_cr = (compute_kl_loss(a1, b1, pad_mask) + compute_kl_loss(a2, b2,
pad_mask)) / 2
        l_aug = (compute_kl_loss(a1, a2, pad_mask) + compute_kl_loss(b1, b2,
pad_mask)) / 2
        return (l_cr + l_aug) / 2

    p_cons = get_consistency(p_logits)
    w_cons = get_consistency(w_logits)

    total_cons = (self.model.config.alpha * p_cons) + ((1 - self.model.config.alpha) *
w_cons)
    loss = outputs["loss"] + CR_LOSS_SCALE * total_cons

    return (loss, outputs) if return_outputs else loss
```

6. Please provide the machine specs and time you used to run your model.
 - CPU (model):
 - GPU (model or N/A): 2 H200 Nvidia GPUs
 - Memory (GB):
 - OS:
 - Train duration: ~33 hours for 5 epochs with pseudo labels, ~10 hours without pseudo labels
 - Inference duration: 25 minutes for single hybrid model, 1h 40 minutes for ensemble 4 models.

7. Anything we should watch out for or be aware of in using your model (e.g. code quirks, memory requirements, numerical stability issues, etc.)?
 - No
8. Did you use any tools for data preparation or exploratory data analysis that aren't listed in your code submission?
 - No
9. How did you evaluate performance of the model other than the provided metric, if at all?
 - For multi-task training, we monitor both **phonetic CER** and **word-level CER/WER**. However, improvements in word-level metrics do not necessarily translate to better phonetic recognition performance.
10. What are some other things you tried that didn't necessarily make it into the final workflow (quick overview)?
 - I experimented with training a hybrid CTC + RNN-T model, but it did not outperform my CTC-based model. This approach likely requires further tuning and more time to fully realize its potential.
 - For the final ensemble, I explored techniques such as ROVER and MBR decoding. However, they did not yield improvements, likely because my trained models lacked sufficient diversity.
11. If you were to continue working on this problem for the next year, what methods or techniques might you try in order to build on your work so far? Are there other fields or features you felt would have been very helpful to have?
 - I would further explore reranking methods, as I was not able to find an effective approach to improve my ensemble. On my fold-0 validation set, my current strategy—selecting the best prediction among four models—only improved CER by about 0.03, whereas the best CER prediction across these predictions suggests a potential improvement of ~0.025. This indicates significant room for better reranking strategies.
 - I would focus on building a more diverse set of models and then apply ensemble techniques such as ROVER or Minimum Bayes Risk (MBR) decoding, as diversity is likely a key factor in making these methods effective.
 - I would also experiment with multiple rounds of pseudo-labeling. So far, I have only used a single round, and iterative refinement could further improve model performance.
12. What simplifications could be made to run your solution faster without sacrificing significant accuracy?

- Instead of combining W2v-BERT 2.0 and WavLM-Large, I could train using only W2v-BERT. While this results in a slight increase in CER, it significantly improves efficiency due to the smaller model size and faster inference time.